

标注“额外内容”的是课上没提到的。

INT102只需要用伪代码实现。

[1] 搜索算法

线性搜索

```
1 public static int linearSearch(int[] array, int number)
2 {
3     for(int i =0;i<array.length;i++)
4         if(array[i]==number)
5             return i;          // 找到了, 返回index
6     return -1;                // 没找到index, 返回-1
7 }
```

二分搜索

递归 Recursive

```
1 public static int binarySearch(int[] array, int number){
2     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
3     else return binarySearch(array,number,0,array.length-1); // 开始二分搜索
4 }
5 private static int binarySearch(int[] array,int number, int left, int right){
6     if(left>right) return -1; // 如果左边大于右边, 直接返回-1。因为越界代表找不到
7     int middleValue = array[(left+right)/2]; // 定义中间值
8     if(number == middleValue) return (left+right)/2; // 如果相等就返回
9     if(number < middleValue) return binarySearch(array,number,left,(left+right)/2-1);
10    // 如果number比中间值小, 就往左边找
11    return binarySearch(array,number,(left+right)/2+1,right); // 如果number比中间值大, 就
    往右边找
12 }
```

循环方法

```
1 public static int binarySearchLoop(int[] array, int number)
2 {
3     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
4     int left = 0;
5     int right = array.length-1;
6     while(left<=right)
7     {
8         int middleValue = array[(left+right)/2];
9         if(number > middleValue){ // 往右边找
10             left = (left+right)/2 +1;
11             continue;
12         }
13         if(number < middleValue){ // 往左边找
14             right = (left+right)/2-1;
15         }
16     }
17 }
```

```

15         continue;
16     }
17     if(number == middlevalue)
18         return (left+right)/2;
19 }
20 return -1;
21 }

```

字符串匹配

给定一个文本 text (String型)和一个模式 pattern。要求找到这个文本 text 内是否含有 pattern。

```

1 public static boolean exist(String text, String pattern)
2 {
3     if(text==null||pattern==null|| text.isEmpty() || pattern.isEmpty()) return false;
4     if(pattern.length()>text.length()) return false;
5
6
7     for(int i = 0;i<text.length();i++)                // 第一层循环
8     {
9         for(int j = 0; j<pattern.length();j++)        // 第二层循环
10        {
11            if(pattern.charAt(j)!=text.charAt(j+i)) break;
12            if(j==pattern.length()-1) return true;
13        }
14    }
15    return false;
16 }

```

[2] 排序算法

选择排序

[看这个视频!](#)

```

1 public static void selectionSort(int[] array){
2     if(array == null||array.length==0) return;
3     for(int i = 0; i<array.length-1;i++)
4     {
5         for(int j=i+1;j<array.length;j++)
6         {
7             if(array[j]<array[i])
8                 swap(array,i,j);
9         }
10    }
11 }
12 private static void swap(int[] array, int indexA, int indexB)
13 {
14     int temp = array[indexA];
15     array[indexA] = array[indexB];

```

```
16     array[indexB] = temp;
17 }
```

冒泡排序

[看这个视频!](#)

```
1 public static void bubbleSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int end = array.length-1; end>=0; end--)
4     {
5         for(int begin = 0; begin<end; begin++)
6         {
7             if(array[begin]>array[begin+1]) swap(array, begin, begin+1);
8         }
9     }
10 }
```

插入排序

[看这个视频!](#)

```
1 public static void insertSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 1; i<array.length; i++)
4     {
5         int j = i-1;
6         while(j>=0 && array[j]>array[j+1])
7         {
8             swap(array, j, j+1);
9             j--;
10        }
11    }
12 }
```

归并排序

[看这个视频!](#)

```
1 public static void mergeSort(int[] arr) {
2     if (arr == null || arr.length < 2) return;
3     mergeSort(arr, 0, arr.length - 1);
4 }
5
6 private static void mergeSort(int[] array, int left, int right) {
7     // 这里左右都要分，分到最小单元之后merge。
8     if (left < right) {
9         int middle = left + (right - left) / 2;
10        mergeSort(array, left, middle);
11        mergeSort(array, middle + 1, right);
12        merge(array, left, middle, right);
13    }
14 }
```

```

13     }
14 }
15
16 private static void merge(int[] array, int left, int middle, int right) {
17     // 第一个数组: left 到 middle
18     // 第二个数组: middle+1 到 right
19     // 现在选出两个数组最小的放在temp中
20     int[] temp = new int[right - left + 1];
21     int indexOfTemp = -1;
22     int firstArrayIndex = left ;
23     int secondArrayIndex = middle+1;
24     while (firstArrayIndex <= middle && secondArrayIndex <= right)
25         if (array[firstArrayIndex] < array[secondArrayIndex])
26             temp[++indexOfTemp] = array[firstArrayIndex++];
27         else
28             temp[++indexOfTemp] = array[secondArrayIndex++];
29
30     // 复制剩余数组到 temp
31     if (firstArrayIndex == middle+1) // 如果first到末尾了
32         while (secondArrayIndex <= right) temp[++indexOfTemp] =
array[secondArrayIndex++];
33     else while (firstArrayIndex <= middle) temp[++indexOfTemp] =
array[firstArrayIndex++];
34
35     // temp替换原来数组
36     indexOfTemp = -1;
37     while (++indexOfTemp < temp.length)
38         array[left + indexOfTemp] = temp[indexOfTemp];
39 }
40
41 public static void main(String[] args) {
42     int[] arr = {51, 13, 10, 64, 34, 5, 32, 21};
43     mergesort(arr);
44     System.out.println(Arrays.toString(arr));
45 }

```

快速排序

额外内容

```

1 public class QuickSort {
2     public static void quickSort(int[] arr)
3     {
4         quickSort(arr, 0, arr.length - 1);
5     }
6     private static void quickSort(int[] arr, int left, int right) {
7         if (left < right) {
8             int pivotPosition = sortArray(arr, left, right);
9             quickSort(arr, left, pivotPosition-1);
10            quickSort(arr, pivotPosition+1, right);
11        }
12    }

```

```

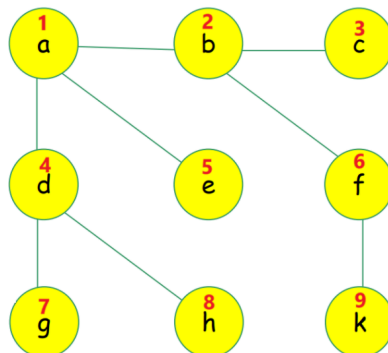
13     private static int sortArray(int[] arr, int left, int right) {
14         int pivot = arr[left]; // 定义基准值是左边第一个元素
15         int i = left;
16         int j = right;
17         // 左边找比基准值大的数，右边找比基准值小的数
18         while (i < j) { // 当左边指针小于等于右边指针，继续走
19             while(i < j && arr[j] > pivot) {j--;} // 从右往左找比基准值小的数
20             arr[i] = arr[j];
21             // 7 7 7 7 9 5 7 4 3
22             // 在这个例子中基准值是7，那么他会越过包括基准值外所有的7然后找到9
23             while(i < j && arr[i] <= pivot) {i++;}
24             arr[j]= arr[i];
25         }
26         arr[i] = pivot;
27         return i;
28     }
29     public static void main(String[] args) {
30         int[] arr = {7,8,3,9,0,6};
31         quickSort(arr);
32         for (int num : arr) {
33             System.out.print(num + " ");
34         }
35     }
36 }

```

[3] 实现BFS和DFS

这个可以看课件

假设我们有如下图



创建图

```

1 public class Graph {
2     private boolean[][] graph; // 这个数组是邻接矩阵，graph[a][b]代表ab点是否联通
3     private boolean[] flag; // 这个数组代表这个点是否被标记

```

```

4 private int size;
5 public Graph(int size) {
6     this.size = size+1;
7     createGraph();
8 }
9 private void createGraph() {
10     graph = new boolean[size][size];
11     flag = new boolean[size];
12 }
13 public void connect(int pointA, int pointB) { // 链接两个点
14     graph[pointA][pointB] = true;
15     graph[pointB][pointA] = true;
16 }
17 public boolean isConnected(int pointA, int pointB) { // 检查两个点是否链接
18     return graph[pointA][pointB];
19 }
20 public ArrayList<Integer> getAdjacentPoints(int pointA) { // 返回A的所有邻接点
21     ArrayList<Integer> adjacentPoints = new ArrayList<>();
22     for (int i = 0; i < size; i++)
23         if (graph[pointA][i])
24             adjacentPoints.add(i);
25     return adjacentPoints;
26 }
27 public void flagThisPoint(int pointA) { // 标记这个点
28     flag[pointA] = true;
29 }
30 public void unFlagThisPoint(int pointA) { // 取消标记这个点
31     flag[pointA] = false;
32 }
33 public boolean isFlagged(int pointA) { // 返回boolean: 这个点是否被标记
34     return flag[pointA];
35 }
36 public ArrayList<Integer> getUnFlaggedPoints(int point) {
37     ArrayList<Integer> unFlaggedPoints = new ArrayList<>();
38     for (int i = 0; i < size; i++)
39     {
40         if(isConnected(point, i)&& !isFlagged(i))
41             unFlaggedPoints.add(i);
42     }
43     return unFlaggedPoints;
44 }
45 }

```

BFS

```

1 public class BFS {
2     public static void main(String[] args) {
3         // 创建大小为9个点的图
4         Graph graph = new Graph(9);
5         // 连接点
6         graph.connect(1,2);
7         graph.connect(2,3);

```

```

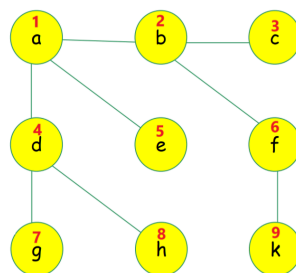
8      graph.connect(2,6);
9      graph.connect(6,9);
10     graph.connect(1,5);
11     graph.connect(1,4);
12     graph.connect(4,8);
13     graph.connect(4,7);
14
15     ArrayList<Integer> bfs = new ArrayList<>(); // 记录BFS的路径
16     ArrayList<Integer> temp = new ArrayList<>(); // 记录待遍历点，相当于缓存
17     temp.add(1);
18     graph.flagThisPoint(0); // 标记待遍历点
19     while(!temp.isEmpty())
20     {
21         int vertex = temp.removeFirst(); // 删除first点，同时赋值到vertex中
22         if(!graph.isFlagged(vertex)) // 如果这个点没有被标记过
23         {
24             bfs.add(vertex);
25             graph.flagThisPoint(vertex);
26         }
27         temp.addAll(graph.getUnFlaggedPoints(vertex));
28     }
29     System.out.println(Arrays.toString(bfs.toArray()));
30 }
31 }

```

输入如下:

```
1 | [1, 2, 4, 5, 3, 6, 7, 8, 9]
```

正好是广度优先搜索的顺序



DFS

```

1  public class DFS{
2      // 创建大小为9个点的图
3      private static Graph graph = new Graph(9);
4      private static ArrayList<Integer> path = new ArrayList();
5      public static void main(String[] args) {
6          graph.connect(1,2);
7          graph.connect(2,3);
8          graph.connect(2,6);
9          graph.connect(6,9);
10         graph.connect(1,5);

```

```

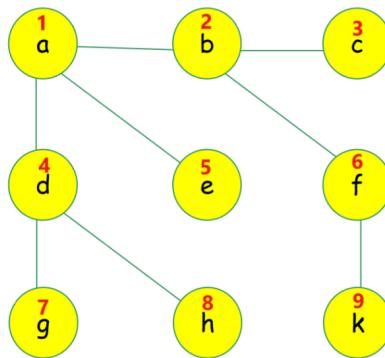
11     graph.connect(1,4);
12     graph.connect(4,8);
13     graph.connect(4,7);
14     path.add(1); // 添加待遍历点
15     graph.flagThisPoint(1); // 标记待遍历点
16     dfs(1); // 开始深搜
17     System.out.println(path);
18 }
19
20 private static void dfs(int vertex) {
21     graph.getUnFlaggedPoints(vertex).forEach(point ->{
22         if(!graph.isFlagged(point)){
23             path.add(point);
24             graph.flagThisPoint(point);
25             dfs(point);
26         }
27     });
28 }
29 }

```

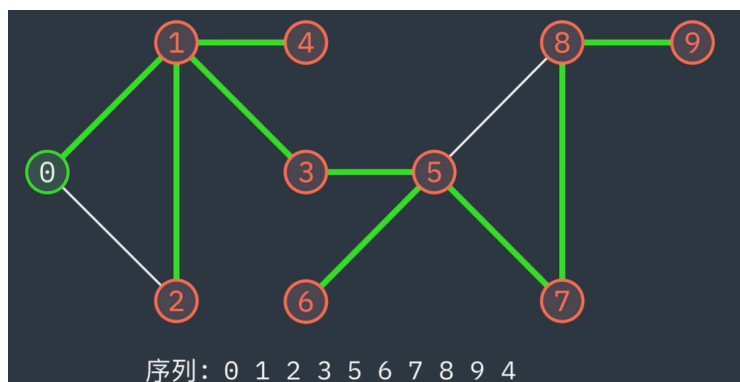
输出

```
1 | [1, 2, 3, 6, 9, 4, 7, 8, 5]
```

确实是深度搜索：



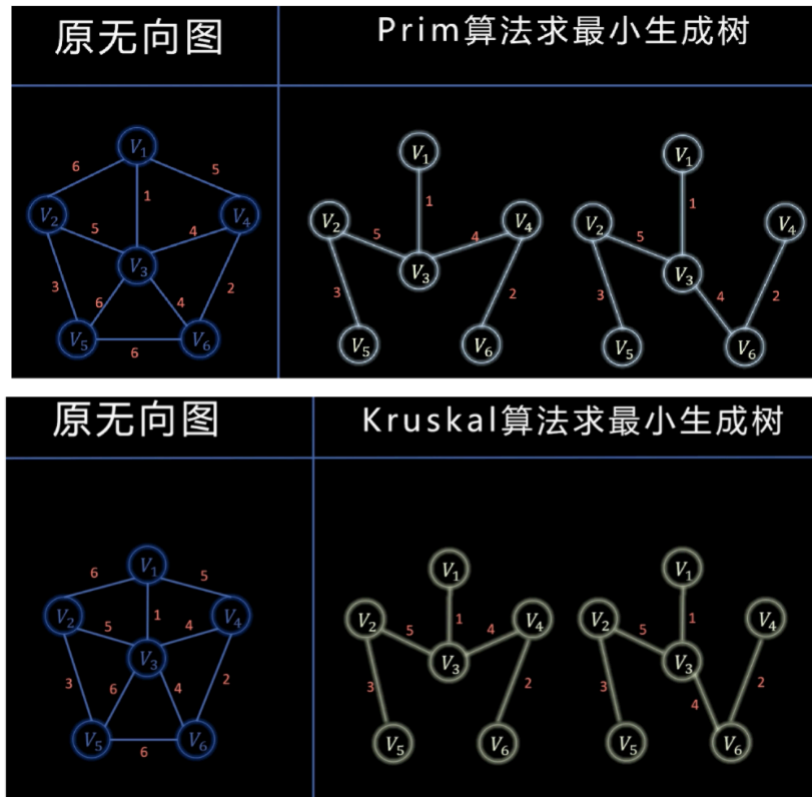
当然换成这个例子，代码也是正确的输出



[4] 图论

最小生成树之 Prim 和 Kruskal 算法

[看这个视频!](#)



首先，我们自己创建一个 `MyMap` 类，这个类包含了图的一些基本的功能...在 `Main` 函数下，我们可以这样写

```
1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.UNDIRECTED_GRAPH); // 无向图
3     // 连接点
4     map.connect("v1", "v2", 6);
5     map.connect("v1", "v3", 1);
6     map.connect("v1", "v4", 5);
7     map.connect("v2", "v3", 5);
8     map.connect("v2", "v5", 3);
9     map.connect("v3", "v4", 4);
10    map.connect("v3", "v5", 6);
11    map.connect("v3", "v6", 4);
12    map.connect("v4", "v6", 2);
13    map.connect("v5", "v6", 6);
14    map.Prim();
15    System.out.println("-----");
16    map.Kruskal();
17 }
```

输出

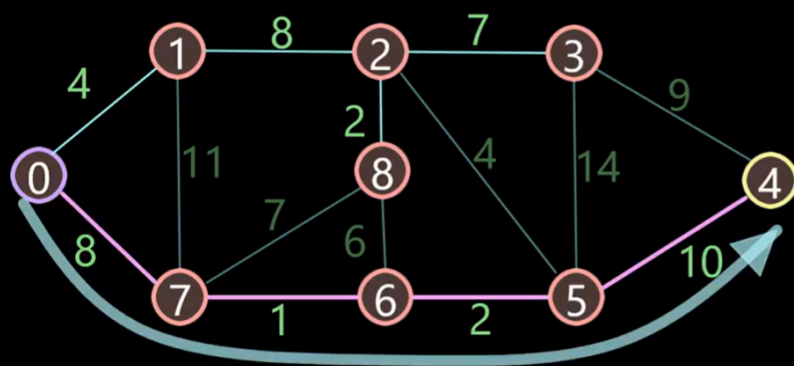
```
1 V6<->V4=2
2 V6<->V3=4
3 V3<->V1=1
4 V3<->V2=5
5 V2<->V5=3
6 -----
7 V3<->V1=1
8 V6<->V4=2
9 V5<->V2=3
10 V6<->V3=4
11 V2<->V3=5
```

Dijkstra 单源最短路径算法

无向图的 Dijkstra

[看这个视频!](#)

1. 每次从未标记的节点中选择距离出发点最近的节点，标记，收录到最优路径集合中。
2. 计算刚加入节点A的邻近节点B的距离（不包含标记的节点），若（节点A的距离+节点A到节点B的边长）< 节点B的距离，就更新节点B的距离和前面点。



	出发点0	前面点
0 ✓	0	
1 ✓	4	0
2 ✓	12	1
3 ✓	19	2
4 ✓	21	5
5 ✓	11	6
6 ✓	9	7
7 ✓	8	0
8 ✓	14	2

```
1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.UNDIRECTED_GRAPH); //无向图的 Dijkstra
3     map.connect("0", "1", 4);
4     map.connect("1", "2", 8);
5     map.connect("2", "3", 7);
6     map.connect("3", "4", 9);
7     map.connect("4", "5", 10);
8     map.connect("5", "6", 2);
9     map.connect("6", "7", 1);
10    map.connect("7", "0", 8);
11
12    map.connect("1", "7", 11);
13    map.connect("2", "8", 2);
```

```

14 map.connect("8", "6", 6);
15 map.connect("7", "8", 7);
16 map.connect("2", "5", 4);
17 map.connect("3", "5", 14);
18
19 map.Dijkstra("0", "4");
20 }

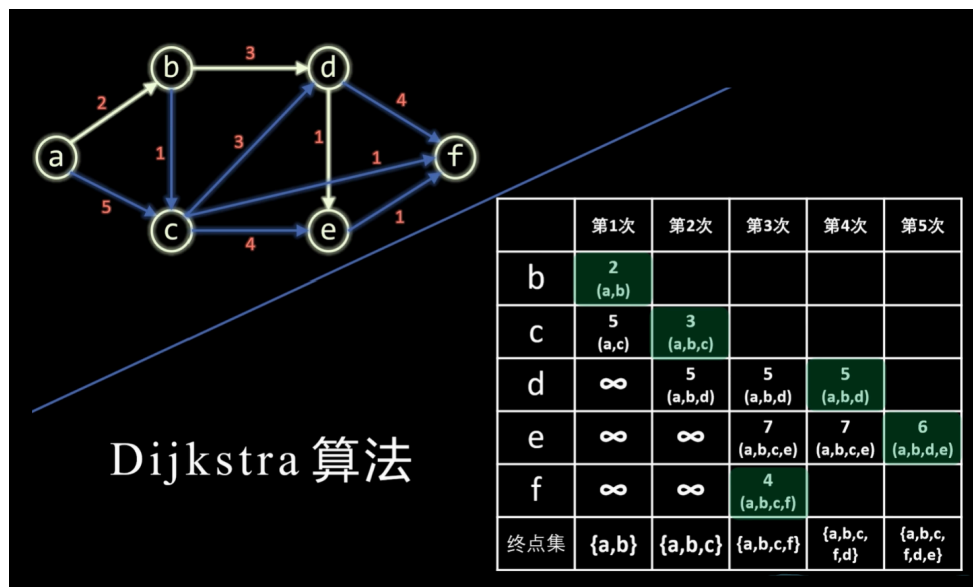
```

输出

```
1 | ["4"]<---21--->["5"]<---11--->["6"]<---9--->["7"]<---8--->["0"]<---0--->
```

有向图的 Dijkstra

[看这个视频!](#)



```

1 public static void main(String[] args) {
2     MyMap map = new MyMap(MyMap.DIRECTED_GRAPH);
3     map.connect("a", "b", 2);
4     map.connect("a", "c", 5);
5     map.connect("b", "c", 1);
6     map.connect("b", "d", 3);
7     map.connect("c", "d", 3);
8     map.connect("c", "e", 4);
9     map.connect("c", "f", 1);
10    map.connect("d", "e", 1);
11    map.connect("d", "f", 4);
12    map.connect("e", "f", 1);
13    map.Dijkstra("a", "f");
14 }

```

MyMap 类

```
1  import java.util.*;
2  import java.util.function.BiConsumer;
3
4
5  public class MyMap {
6      public static final int DIRECTED_GRAPH = 1; // 有向图
7      public static final int UNDIRECTED_GRAPH = 2; // 无向图
8      private static final int INFINITY = Integer.MAX_VALUE;
9      private final HashMap<Node, Integer> map = new HashMap<>(); // 整个图，记录了点和权重
10     private final HashSet<HashSet<String>> set = new HashSet<>(); // [无向图]分组
11     private final HashMap<String, LinkedHashSet<String>> adjList = new HashMap<>(); //
    [有向图]邻接表，参数1是头，参数2是邻接表
12     private final HashMap<String, Boolean> nodes = new HashMap<>(); // 表示点，以及有没有
    被标记
13     private final HashMap<Node, Boolean> edge = new HashMap<>(); // 表示边，以及边是否被
    标记
14     private int totalNodesNumber;
15     private int flaggedNodesNumber;
16     private int totalEdgesNumber;
17     private int flaggedEdgesNumber;
18     private final int MODE;
19
20     public MyMap(int GRAPH_MODE) {
21         this.MODE = GRAPH_MODE;
22         if (MODE != DIRECTED_GRAPH && MODE != UNDIRECTED_GRAPH) {
23             throw new IllegalArgumentException("必须是有向图或者无向图之一");
24         }
25     } // 创建的时候必须指定图的类型
26
27     public boolean connect(String a, String b, int weight) {
28         switch (MODE) {
29             case DIRECTED_GRAPH -> {
30                 return connectDirectedGraph(a, b, weight);
31             }
32             case UNDIRECTED_GRAPH -> {
33                 return connectUndirectedGraph(a, b, weight);
34             }
35         }
36         return false;
37     }
38
39     private boolean connectDirectedGraph(String a, String b, int weight) {
40         // 如果这个图已经记录了ab直接连通，那么返回，连接失败
41         if (map.containsKey(new Node(a, b))) return false;
42
43         // 连接这两个点，现在他们连通了
44         map.put(new Node(a, b), weight);
45
46         // 邻接表
47         // 先检查邻接表有没有这个点：
```

```

48 // 如果存在，直接把B添加到邻接表后头
49 // 否则，就创建点A，然后添加点B
50 boolean containsNodeA = adjList.containsKey(a);
51 if (containsNodeA) {
52     adjList.get(a).add(b);
53 } else {
54     LinkedHashSet<String> nodes = new LinkedHashSet<>();
55     nodes.add(a);
56     nodes.add(b);
57     adjList.put(a, nodes);
58 }
59 // 记录点数量
60 if (!nodes.containsKey(a)) ++totalNodesNumber;
61 if (!nodes.containsKey(b)) ++totalNodesNumber;
62 // 添加点 node
63 nodes.put(a, false);
64 nodes.put(b, false);
65 // 添加边 edge
66 edge.put(new Node(a, b), false);
67 // 记录边数量 totalEdgeNumber
68 totalEdgesNumber++;
69
70 return true;
71 } // 有向图
72
73 private boolean connectUndirectedGraph(String a, String b, int weight) {
74
75     // 如果这个图已经记录了ab直接连通，那么返回，连接失败
76     if (map.containsKey(new Node(a, b)) || map.containsKey(new Node(b, a))) return
false;
77
78     // 连接这两个点
79     map.put(new Node(a, b), weight);
80     map.put(new Node(b, a), weight);
81
82     // 给他们划分组别，如果连通就放在一组，否则就新开一个组
83     // 有一些情况：
84     // 点A和B都被记录，而且都在同一个组：不需要分组
85     // 点A和B都被记录，但不在同一个组：合并他们的组
86     // 点A和B其中一个被记录：直接添加另一个到当前组
87     // 点A和B都没有被记录：创建新组
88     boolean containsNodeA = nodes.containsKey(a);
89     boolean containsNodeB = nodes.containsKey(b);
90     HashSet<String> setA = new HashSet<>();
91     HashSet<String> setB = new HashSet<>();
92     boolean setAFound = false;
93     boolean setBFound = false;
94     boolean inSameSet = false;
95     // 点A和B都被记录
96     if (containsNodeA && containsNodeB) {
97         Iterator<HashSet<String>> iterator = set.iterator();
98         while (iterator.hasNext()) {
99             HashSet<String> sets = iterator.next();

```

```

100         boolean setContainsA = sets.contains(a);
101         boolean setContainsB = sets.contains(b);
102         // 而且都在同一个组：不需要分组
103         if (setContainsA && setContainsB) break;
104         // 但不在同一个组：合并他们的组
105         if (setContainsA) {
106             setA = sets;
107             setAFound = true;
108         }
109         if (setContainsB) {
110             setB = sets;
111             setBFound = true;
112         }
113         if (setAFound && setContainsB) {
114             setA.addAll(sets);
115             iterator.remove();
116             break;
117         }
118         if (setBFound && setContainsA) {
119             setB.addAll(sets);
120             iterator.remove();
121             break;
122         }
123     }
124 }
125 // 点A和B其中一个被记录
126 if (containsNodeA || containsNodeB) {
127     // 直接添加另一个到当前组
128     for (HashSet<String> sets : set) {
129         if (sets.contains(a)) {
130             sets.add(b);
131             break;
132         }
133         if (sets.contains(b)) {
134             sets.add(a);
135             break;
136         }
137     }
138 }
139 // 点A和B都没有被记录：创建新组
140 if (!containsNodeA && !containsNodeB) {
141     HashSet<String> newHashSet = new HashSet<>();
142     newHashSet.add(a);
143     newHashSet.add(b);
144     set.add(newHashSet);
145 }
146
147 // 记录点数量
148 if (!containsNodeA) totalNodesNumber++;
149 if (!containsNodeB) totalNodesNumber++;
150
151
152 // 添加点

```

```

153     nodes.put(a, false);
154     nodes.put(b, false);
155
156     // 添加边
157     edge.put(new Node(a, b), false);
158     edge.put(new Node(b, a), false);
159
160     // 记录边数量
161     if (!(containsNodeA && containsNodeB)) totalEdgesNumber++;
162
163     return true;
164 } // [无向图]链接这两个点，以及他们的权重，返回一个boolean表示他们是否添加成功
165
166 public boolean isConnected(Node node) {
167
168     switch (MODE) {
169         case DIRECTED_GRAPH -> {
170             // 如果其中一个点没有被记录过，自然也不是连接的
171             if (!nodes.containsKey(node.A) || !nodes.containsKey(node.B)) return
false;
172
173             // 如果这两个点是直接连接，就直接返回true
174             if (isDirectConnected(node.A, node.B)) return true;
175             // 现在，要判断这两个点是否间接连接
176             // 遍历邻接表，从a开始，一直找，看看能否找到b
177             // 遍历过的点需要标记
178             HashSet<String> visited = new HashSet<>();
179             LinkedHashSet<String> nodesShouldCheckItsAdjList = new LinkedHashSet<>
();
180
181             LinkedHashSet<String> adjListOfA = adjList.get(node.A);
182             // 一直找点a和b是否连接
183             while (visited.size() != getTotalNodeNumber()) {
184                 // 遍历a点的邻接表
185                 for (String nodes : adjListOfA) {
186                     // 如果当前点nodes和目标点b相等就返回 true
187                     if (nodes.equals(node.B)) return true;
188                     // 否则，把这个nodes标记为已访问，然后加入
nodesShouldCheckItsAdjList
189                     // 意思是下一次需要访问这个点的邻接表
190                     else {
191                         if (nodes.equals(node.A)) visited.add(nodes); // 如果这个表
是表头，就不再访问
192
193                         // 如果nodes有邻接表而且这个点还没有被访问
194                         if (adjList.containsKey(nodes) &&
!visited.contains(nodes))
nodesShouldCheckItsAdjList.add(nodes);
195                     }
196                 }
197                 adjListOfA = adjList.get(nodesShouldCheckItsAdjList.removeLast());
198             }
199             return false;
200         case UNDIRECTED_GRAPH -> {

```

```

201         // 如果其中一个点没有被记录过，自然也不是连接的
202         if (!nodes.containsKey(node.A) || !nodes.containsKey(node.B)) return
false;
203         // 如果这两个点是直接连接，就直接返回true
204         if (isDirectConnected(node.A, node.B)) return true;
205         // 现在，要判断这两个点是否间接连接（也就是检查是不是在一个组）
206         for (HashSet<String> sets : set) {
207             if (sets.contains(node.A)) return sets.contains(node.B);
208             if (sets.contains(node.B)) return sets.contains(node.A);
209         }
210     }
211 }
212
213     return false;
214 }
215
216 public boolean isConnected(String a, String b) {
217     return isConnected(new Node(a, b));
218 } // 检查这两个点是不是直接/间接连接的
219
220 public boolean isDirectConnected(String a, String b) {
221     return map.containsKey(new Node(a, b));
222 } // 检查是否直接连接的
223
224 public void unFlagAllPoint() {
225     nodes.forEach((a, b) -> {
226         b = false;
227     });
228     flaggedNodesNumber = 0;
229 } // 取消标记所有点
230
231
232 public void flagAllPoint() {
233     nodes.forEach((a, b) -> {
234         b = true;
235     });
236     flaggedNodesNumber = totalNodesNumber;
237 } // 标记所有点
238
239
240 public void flagPoint(String point) {
241     if (nodes.containsKey(point)) {
242         nodes.put(point, true);
243         flaggedNodesNumber++;
244     }
245 } // 标记某个点
246
247
248 public void unFlagPoint(String point) {
249     if (nodes.containsKey(point)) {
250         nodes.put(point, false);
251         flaggedNodesNumber--;
252     }

```

```

253     } // 取消标记某个点
254
255     public void flagAllEdge() {
256         edge.forEach((a, b) -> {
257             b = true;
258         });
259     } // 标记所有边
260
261     public void unFlagAllEdge() {
262         edge.forEach((a, b) -> {
263             b = false;
264         });
265     } // 取消标记所有边
266
267     public void flagEdge(String pointA, String pointB) {
268         if (map.containsKey(new Node(pointA, pointB))) {
269             edge.put(new Node(pointA, pointB), true);
270             flaggedEdgesNumber++;
271         }
272     } // 标记某条边
273
274     public void unFlagEdge(String pointA, String pointB) {
275         if (map.containsKey(new Node(pointA, pointB))) {
276             edge.put(new Node(pointA, pointB), false);
277             flaggedEdgesNumber--;
278         }
279     } // 取消标记某条边
280
281     public boolean isAllFlagged() {
282         return flaggedNodesNumber == totalNodesNumber;
283     } // 是否全部点都标记了
284
285
286     public boolean isAllUnFlagged() {
287         return flaggedNodesNumber == 0;
288     } // 是否全部点都没标记
289
290     /**
291      * 返回这个点所有的直接连接点，以及对应的权重
292      * 如果是无向图，返回所有连接点
293      * 如果是有向图，返回邻接表
294      */
295     public HashMap<String, Integer> getAllNearbyPoint(String point) {
296         // 例如getAllNearbyPoint("A")
297         // 返回一个HashMap，包含<String, Integer> 第一个参数是点名称，第二个是权重
298         HashMap<String, Integer> returnMap = new HashMap<>();
299         nodes.forEach((a, b) -> {
300             if (map.containsKey(new Node(point, a))) returnMap.put(a, map.get(new
Node(point, a)));
301         });
302         return returnMap;
303     }
304

```

```

305     public HashSet<String> getAllUnFlaggedPoint() {
306         HashSet<String> returnSet = new HashSet<>();
307         nodes.forEach((a, b) -> {
308             if (!b) returnSet.add(a);
309         });
310         return returnSet;
311     } // 返回所有未标记点
312
313     public HashSet<String> getAllFlaggedPoint() {
314         HashSet<String> returnSet = new HashSet<>();
315         nodes.forEach((a, b) -> {
316             if (b) returnSet.add(a);
317         });
318         return returnSet;
319     } // 返回所有标记点
320
321     public HashMap<Node, Integer> getAllUnFlaggedEdge() {
322         HashMap<Node, Integer> returnSet = new HashMap<>();
323         edge.forEach((a, b) -> {
324             if (!b) returnSet.put(a, map.get(a));
325         });
326         return returnSet;
327     } // 返回所有未标记边
328
329     public HashMap<Node, Integer> getAllFlaggedEdge() {
330         HashMap<Node, Integer> returnSet = new HashMap<>();
331         edge.forEach((a, b) -> {
332             if (b) returnSet.put(a, map.get(a));
333         });
334         return returnSet;
335     } // 返回所有标记边
336
337     public boolean isEmpty() {
338         return nodes.isEmpty();
339     } // map是否为空
340
341     public String getClosestPointName(String objectPoint) {
342         if (!nodes.containsKey(objectPoint)) return null;
343
344         String returnPoint = null;
345         int weight = Integer.MAX_VALUE;
346         for (String nodes : nodes.keySet()) {
347             if (map.containsKey(new Node(objectPoint, nodes)) && weight > map.get(new
Node(objectPoint, nodes))) {
348                 weight = map.get(new Node(objectPoint, nodes));
349                 returnPoint = nodes;
350             }
351         }
352         return returnPoint;
353     } // 返回这个点周围与他最小权重的点的名字
354
355
356     public int getClosestPointweight(String objectPoint) {

```

```

357         if (!nodes.containsKey(objectPoint)) return 0;
358
359         String returnPoint = null;
360         int weight = Integer.MAX_VALUE;
361         for (String nodes : nodes.keySet()) {
362             if (map.containsKey(new Node(objectPoint, nodes)) && weight > map.get(new
Node(objectPoint, nodes))) {
363                 weight = map.get(new Node(objectPoint, nodes));
364                 returnPoint = nodes;
365             }
366         }
367         return weight;
368     } // 返回这个点周围与他最小权重的点的权重
369
370
371     public void Prim() {
372         if (isEmpty()) return; // 当前集合是空的，就返回
373         if (set.size() != 1) return; // 当前的图存在多个集，返回（不能生成树）
374
375
376         String node = nodes.keySet().toArray(new String[0])[0]; // 获取任意节点
377         flagPoint(node); // 标记这个节点
378
379         // 当不是所有点都被标记，继续执行
380         while (!isAllFlagged()) {
381             HashSet<String> allFlaggedPoint = getAllFlaggedPoint();
382             HashSet<String> allUnFlaggedPoint = getAllUnFlaggedPoint();
383             String newPoint = null; // 新的等待加入的点
384             String selectedPoint = null; // 新的等待加入的点所连接的点
385             int minWeight = Integer.MAX_VALUE;
386             for (String flaggedPoint : allFlaggedPoint) {
387                 for (String unFlaggedPoint : allUnFlaggedPoint) {
388                     // 现在遍历nodes
389                     // nodes是点亮的点
390                     // 需要找到距离最近的非点亮点
391                     if (map.containsKey(new Node(flaggedPoint, unFlaggedPoint)) &&
getWeight(flaggedPoint, unFlaggedPoint) < minWeight) {
392                         minWeight = getWeight(flaggedPoint, unFlaggedPoint);
393                         newPoint = unFlaggedPoint;
394                         selectedPoint = flaggedPoint;
395                     }
396                 }
397             }
398             flagPoint(newPoint);
399             System.out.println(selectedPoint + "<->" + newPoint + "=" +
getWeight(selectedPoint, newPoint));
400         }
401     } // Prim 算法
402
403     public int getWeight(String pointA, String pointB) {
404         if (pointA.equals(pointB)) return 0;
405         return map.get(new Node(pointA, pointB));
406     } // 返回两个点的权重

```

```

407
408
409
410     private int getTotalNodeNumber() {
411         return totalNodesNumber;
412     } // 获取点的数量
413
414
415     private int getTotalEdgeNumber() {
416         return flaggedEdgesNumber;
417     } // 获取边的数量
418
419     public void kruskal() {
420         if (isEmpty()) return; // 当前集合是空的，就返回
421         if (set.size() != 1) return; // 当前的图存在多个集，返回（不能生成树）
422
423         int nodesToBeConnect = nodes.size() - 1;
424         List<Map.Entry<Node, Integer>> weightList = new ArrayList<>(map.entrySet());
425         // 升序，这样我们可以从尾部取出最小的 Entry <点, 边>
426         weightList.sort(Map.Entry.comparingByValue(Comparator.reverseOrder()));
427
428         MyMap tempMap = new MyMap(MyMap.UNDIRECTED_GRAPH);
429         while (nodesToBeConnect > 0) {
430             Node node = weightList.getLast().getKey();
431             // 获取到没有被标记的边
432             // 如果这两个点是连通的，我们就不需要这条边
433             while (tempMap.isConnected(node)) {
434                 weightList.removeLast();
435                 weightList.removeLast();
436                 node = weightList.getLast().getKey();
437             }
438             tempMap.connect(node.A, node.B, 0);
439             nodesToBeConnect--;
440             System.out.println(weightList.getLast().getKey().A + "<->" +
weightList.getLast().getKey().B + "=" + weightList.getLast().getValue());
441             weightList.removeLast(); // 移除这条边
442             weightList.removeLast();
443         }
444     }
445
446     public void Dijkstra(String a, String b) {
447         if (!nodes.containsKey(a) || !nodes.containsKey(b)) return;
448         int leftNode = getTotalNodeNumber();
449         // 初始包含所有点
450         HashSet<String> unFlaggedPoint = new HashSet<>(nodes.keySet());
451         // chart是图标
452         // 如果一个点没有在chart中，证明它的权重是Infinity
453         // 如果一个点在chart中但是没有在unFlaggedPoint中，那就是这个点被标记
454         HashMap<String, Entry> chart = new HashMap<>();
455
456         String currentPoint = a;
457         // 设置自己到自己的距离为0
458         chart.put(a, new Entry(0, null));

```

```

459         unFlaggedPoint.remove(a); // 标记当前点
460         while(leftNode>0)
461         {
462             // 获取当前点的所有连接点
463             HashMap<String, Integer> allNearbyPoint = getAllNearbyPoint(currentPoint);
464             // 更新每一个nearByPoint
465             for (String s : allNearbyPoint.keySet()) {
466                 if(!unFlaggedPoint.contains(s)) continue; // 如果被标记了就跳过这个点
467                 // 如果这个点是INF, 那么就更新
468                 if(!chart.containsKey(s))
469                 {
470                     chart.put(s, new Entry(getWeight(currentPoint, s) +
471 chart.get(currentPoint).weight, currentPoint));
472                     continue;
473                 }
474                 // 现在这个点不是INF, 让我们更新它
475                 // 如果 chart(s) > currentPoint 到 s 的值 + chart(current)
476                 // 那就更新 chart(s)
477                 if(chart.get(s).weight > getWeight(currentPoint, s) +
478 chart.get(currentPoint).weight)
479                 {
480                     chart.put(s, new Entry(getWeight(currentPoint, s) +
481 chart.get(currentPoint).weight, currentPoint));
482                     continue;
483                 }
484             }
485             unFlaggedPoint.remove(currentPoint);
486             // 现在更新完了, 需要从 未标记的 找到最小值
487             String minPoint = null;
488             int minWeight = Integer.MAX_VALUE;
489             for (String s : chart.keySet()) {
490                 if(!unFlaggedPoint.contains(s)) continue; // 被标记了就继续
491                 if(chart.get(s).weight < minWeight)
492                 {
493                     minWeight = chart.get(s).weight;
494                     minPoint = s;
495                 }
496             }
497             currentPoint = minPoint;
498             unFlaggedPoint.remove(currentPoint); // 更换点, 然后标记为访问
499             leftNode--;
500         }
501         String iterateNode = b;
502         while(iterateNode != null)
503         {
504             System.out.print("[\""+iterateNode+"\"]"+"<---
505 "+chart.get(iterateNode).weight+"--->");
506             iterateNode = chart.get(iterateNode).prePoint;
507         }
508     }
509     public boolean isPointFlagged(String point) {
510         return nodes.get(point);
511     }

```

```

508
509     public boolean isEdgeFlagged(String pointA, String pointB) {
510         return isEdgeFlagged(new Node(pointA, pointB));
511     }
512
513     public boolean isEdgeFlagged(Node node) {
514         return edge.get(node);
515     }
516
517     public static class Node {
518         private String A;
519         private String B;
520
521         public Node(String a, String b) {
522             A = a;
523             B = b;
524         }
525
526         public String getA() {
527             return A;
528         }
529
530         public void setA(String a) {
531             A = a;
532         }
533
534         public String getB() {
535             return B;
536         }
537
538         public void setB(String b) {
539             B = b;
540         }
541
542         @Override
543         public boolean equals(Object o) {
544             if (o == null || getClass() != o.getClass()) return false;
545             Node node = (Node) o;
546             return Objects.equals(A, node.A) && Objects.equals(B, node.B);
547         }
548
549         @Override
550         public int hashCode() {
551             return Objects.hash(A, B);
552         }
553
554         @Override
555         public String toString() {
556             return "Node{" + "A='" + A + '\'' + ", B='" + B + '\'' + '}';
557         }
558     } // 公开内部类
559
560     // String: 当前点

```

```

561 // 第二个 Entry : 权重, 以及上一个点
562 // 其中权重指的是 a 到 当前点的最优距离
563 private class Entry{
564     public int weight;
565     public String prePoint;
566
567     public Entry(int weight, String prePoint) {
568
569         this.weight = weight;
570         this.prePoint = prePoint;
571     }
572
573     public int getWeight() {
574         return weight;
575     }
576
577     public void setWeight(int weight) {
578         this.weight = weight;
579     }
580
581     public String getPrePoint() {
582         return prePoint;
583     }
584
585     public void setPrePoint(String prePoint) {
586         this.prePoint = prePoint;
587     }
588
589     @Override
590     public boolean equals(Object o) {
591         if (o == null || getClass() != o.getClass()) return false;
592         Entry entry = (Entry) o;
593         return weight == entry.weight && Objects.equals(prePoint, entry.prePoint);
594     }
595
596     @Override
597     public int hashCode() {
598         return Objects.hash(weight, prePoint);
599     }
600 }
601 }
602

```

[5] 树

额外内容

二叉搜索树

```

1 public class BinarySearchTree {

```

```

2
3 private static ArrayList<Integer> tempList = new ArrayList<>(); // 返回数组
4 private Nodes treeHead; // 一个bst对象有一个树头，初始为null
5
6 BinarySearchTree() { // 空参构造
7 }
8
9
10 // 添加数据
11 public void add(int number) {
12     Nodes search = treeHead; // 初始search是树头，以后就操作search即可
13
14     if (search == null) { // 如果search是空的就创建新节点
15         treeHead = new Nodes();
16         treeHead.data = number;
17     } else { // 非空情况
18         boolean flag = false;
19         while (true) {
20             if (number == search.data) return; // 相等的数据就不执行add操作。
21             if (number < search.data) { // 如果number 小于当前search数据，就往左边找
22                 if (search.left == null) {
23                     flag = false; // 标记flag=false 意思是要添加的数据是search的
左节点
24                     break;
25                 } // 如果是尾节点就断开
26                 search = search.left;
27                 continue;
28             }
29             if (number > search.data) { // 如果number 大于当前search数据，往右找
30                 if (search.right == null) { // 如果是尾节点就break
31                     flag = true; // 标记flag=true意思是要添加的数据是search
的右节点
32                     break;
33                 }
34                 search = search.right;
35             }
36         }
37     }
38
39     Nodes newNode = new Nodes();
40     newNode.middle = search; // 新点的父节点是search
41     newNode.data = number; // 新点数据
42
43
44     if (flag) // flagPoint = true, 意味着添加的数据在search右侧。
45     {
46         search.right = newNode;
47     } else { // flagPoint = false, 意味着添加的数据在search左侧。
48         search.left = newNode;
49     }
50 }
51 }
52

```

```

53 // 返回ArrayList;
54 public ArrayList<Integer> getTree() {
55     tempList = new ArrayList<>();
56     getTree(treeHead);
57     return tempList;
58 }
59
60 private void getTree(Nodes nodes) { // 递归的方式中序遍历
61     if (treeHead == null) return;
62     if (nodes.left != null) getTree(nodes.left);
63     tempList.add(nodes.data);
64     if (nodes.right != null) getTree(nodes.right);
65 }
66
67 // 查询是否包含某个点
68 public boolean contains(int number) {
69     return contains(treeHead, number);
70 }
71
72 private boolean contains(Nodes node, int number) {
73     if (node == null) return false;
74     if (number == node.data) return true;
75     if (number < node.data) return contains(node.left, number);
76     // if(number > node.data)
77     return contains(node.right, number);
78 }
79
80 // 删除节点操作
81 public boolean delete(int number) {
82
83     if (treeHead == null) return false; // 如果当前对象没有树，那么直接返回false，删除失
败。
84     Nodes search = treeHead; // 让search从头节点开始操作；search代表要删除的
点
85     boolean flag = false; // search是上一个点的左边还是右边：false ->
left; true->right
86     while (search != null) { // 找到要删除的点
87         if (number == search.data) break; // 找到就break
88         if (number > search.data) { // 如果number大于当前search，那么往右边找。
89             flag = true; // flagPoint = true代表往右找
90             search = search.right;
91             continue; // 一定要continue否则会下面if冲突
92         }
93         if (number < search.data) {
94             flag = false; // 往左边找，标记flag
95             search = search.left;
96         }
97     }
98     // 如果不存在就返回false
99     if (search == null) return false;
100
101     // 如果要删除的是头节点，而且该对象仅有头节点，那么头节点为null即可
102     if (search == treeHead && treeHead.left == null && treeHead.right == null) {

```

```

103         treeHead = null;
104         return true;
105     }
106
107
108
109     // 情况1: 叶子节点
110     if (search.left == null && search.right == null) {
111         if (flag) { // 删除右边
112             search.middle.right = null;
113         } else { // 删除左边
114             search.middle.left = null;
115         }
116
117         search.middle = null;
118         return true;
119     }
120
121
122     // 情况2: 只有右子树
123     if (search.left == null && search.right != null) {
124         //如果是头节点， 直接让头节点指向下一个即可
125         if (search == treeHead) {
126             treeHead = treeHead.right;
127             treeHead.middle = null; //取消引用，直接回收内存
128         } else if (flag) { // 右边
129             search.middle.right = search.right;
130             search.right.middle = search.middle;
131
132         } else { // 左边
133             search.middle.left = search.right;
134             search.right.middle = search.middle;
135         }
136
137         search.middle = null;
138         search.left = null;
139         search.right = null;
140         return true;
141     }
142
143     // 情况3: 只有左子树
144     if (search.left != null && search.right == null) {
145         if (search == treeHead) {
146             treeHead = treeHead.left;
147             treeHead.middle = null;
148         } else if (flag) { // 右边
149             search.middle.right = search.left;
150             search.left.middle = search.middle;
151         } else { // 左边
152             search.middle.left = search.left;
153             search.left.middle = search.middle;
154         }
155
156         search.middle = null;

```

```

156         search.left = null;
157         search.right = null;
158         return true;
159     }
160
161     // 情况4: 既有左子树也有右子树。
162     if (search.left != null && search.right != null) {
163         boolean sign = false;    // sign用与判断search节点在上一个节点的左还是右。默认
false= 左, 如果true则为右
164         Nodes temp = search;    // temp是search左子树中最大的点
165         temp = temp.left;
166         while (temp.right != null) { // 让temp作为search左子树中最大的节点
167             temp = temp.right;
168             sign = true;
169         }
170
171         // 如果 temp 有左节点 , 就晋升左节点到 temp 位置
172         Nodes tempFather = temp.middle; // 记录temp的父节点
173         Nodes leftTree = temp.left; // 记录temp左子树
174
175         if (leftTree != null) leftTree.middle = null;    // 左子树和temp断开
176         temp.left = null;    // temp左右子树都是null (temp不可能有右子树)
177
178         // 断开temp和父节点
179         if (sign) {
180             temp.middle.right = null;
181         } else {
182             temp.middle.left = null;
183         }
184         temp.middle = null;
185
186         // 让左子树晋升
187         if (leftTree != null) {
188             leftTree.middle = tempFather;
189             if (sign) {
190                 tempFather.right = leftTree;
191             } else {
192                 tempFather.left = leftTree;
193             }
194         }
195
196         // 现在要让temp和search换位
197         temp.middle = search.middle;
198         temp.left = search.left;
199         temp.right = search.right;
200
201         // 如果search这个点不是treeHead, 就更改其middle
202         if (search.middle != null) {
203             if (flag) { // flagPoint 代表search是上一个点的左边还是右边, true代表search
在 上一个点的右边
204                 search.middle.right = temp;
205             } else {    // search在上一个点的左边
206                 search.middle.left = temp;

```

```

207         }
208     }
209
210     // 让search的左右子树指向temp
211     if (search.left != null) search.left.middle = temp;
212     if (search.right != null) search.right.middle = temp;
213
214     // 如果更改的是树头，就让树头 = temp;
215     if (search == treeHead) treeHead = temp;
216
217
218     return true;
219 }
220
221 return false;
222 }
223
224 private static class Nodes {
225     Nodes left;
226     Nodes middle;
227     Nodes right;
228     int data;
229 }
230 }

```

AVL 树

```

1  import java.util.ArrayList;
2  import java.util.Stack;
3
4  /*
5   * 1. 点A的平衡因子定义如下：A的左子树高度 - A的右子树高度
6   * 如果子树为null，那么计算而言，高度视为0
7   * 2. 失衡节点因子=2，其左节点因子=1；对失衡节点进行左旋    LL
8   * 失衡节点因子=2，左节点因子=-1；左节点左旋后，节点右旋    LR
9   * 失衡节点因子=-2，其右节点因子=-1；对失衡节点进行左旋    RR
10  * 失衡节点因子=-2，其右节点因子=1；右节点右旋，节点左旋    RL
11  * */
12  public class AVLTree {
13      private Nodes treeHead; // 树头
14
15
16
17
18      public AVLTree() {
19          // 无参构造
20      }
21      private static class Nodes{
22          int key; // 当前节点值
23          Nodes left; // 左子树
24          Nodes right; // 右子树

```

```

24     Nodes middle; // 上一级
25     int height = 1; // 把当前节点作为一棵树，这棵树的高度
26 } // 节点
27
28
29 // 插入
30 public boolean insert(int number){
31     // 树头不存在
32     if(treeHead == null)
33     {
34         treeHead = new Nodes();
35         treeHead.key = number;
36         return true;
37     }
38     // 树头存在
39     // search是作为临时搜寻的。其作用是充当待插入节点的父节点
40     Nodes search = treeHead;
41     // 寻找插入节点的父节点
42     while(search!=null)
43     {
44         // 如果这个值存在，就不插入数据，返回false
45         if(number == search.key) return false;
46
47         // 如果数字大于当前节点，就需要往右边找
48         // 如果需要往右找，但是右边是null，直接break
49         if(number>search.key)
50         {
51             if(search.right==null) break;
52             search = search.right;
53             continue;
54         }
55         // 如果数字小于当前节点，就需要往左边找
56         // 同理如果左边是null，直接break
57         if(number<search.key)
58         {
59             if(search.left==null) break;
60             search = search.left;
61         }
62     }
63
64     // 现在的search就待插入节点的父节点。
65
66     // 什么时候需要更新父节点的高度？
67     //     当数据插入父节点的子节点，例如插入父节点的左边，
68     //     但是父节点的另一边（右边）是空的，那么父节点就要更新。
69     //     父节点的父节点就要判断谁大，一路比下去
70
71     // 应当插入左节点的情况
72     // 先申请一个节点，赋值，作为左节点
73     // 左节点指向search这个父节点
74     // 如果这个树的高度变了（另一侧为null），就更新父节点search的高度，并且迭代上去
75     if(number<search.key)
76     {

```

```

77         search.left = new Nodes();
78         search.left.key = number;
79         search.left.middle = search;
80         if(search.right==null) updateHeight(search);
81         executeRotateOperation(search.left);
82         return true;
83     }
84
85     // 这里就是应当插入右节点的情况
86     // 申请一个节点，赋值，作为右节点
87     // 右节点指向search这个父节点
88     // 同理如果树增长了就更新
89     if(number>search.key)
90     {
91         search.right = new Nodes();
92         search.right.key = number;
93         search.right.middle = search;
94         if(search.left==null) updateHeight(search);
95         executeRotateOperation(search.right);
96         return true;
97     }
98     return false;
99 }
100 private void executeRotateOperation(Nodes node){
101     if(node == null) return;
102     // 一直寻找到失衡节点(往插入节点的父节点找)
103     while((node!=null&&Math.abs(getBalanceFactor(node))<=1)
104     {
105         node = node.middle;
106     }
107     if(node == null) return;
108     /*if(Math.abs(getBalanceFactor(node))>2)
109     {
110         system.out.println("Error: diff abs is larger than 2. Diff = " +
111         Math.abs(getBalanceFactor(node.left) - getBalanceFactor(node.right)));
112         return;
113     }*/
114     // LL
115     if(getBalanceFactor(node)==2&&getBalanceFactor(node.left)==1)
116     {
117         rightRotate(node);
118         return;
119     }
120     // RR
121     if(getBalanceFactor(node)==-2&&getBalanceFactor(node.right)==-1)
122     {
123         leftRotate(node);
124         return;
125     }
126     // LR
127     if(getBalanceFactor(node)==2&&getBalanceFactor(node.left)==-1)
128     {
129         leftRotate(node.left);

```

```

129         rightRotate(node);
130         return;
131     }
132     // RL
133     if(getBalanceFactor(node)==-2&&getBalanceFactor(node.right)==1)
134     {
135         rightRotate(node.right);
136         leftRotate(node);
137         return;
138     }
139     // System.out.println("Error: didn't find suitable rotation operation.");
140 }
141 private static void updateHeight(Nodes search){
142     while(search!=null){
143         int leftHeight = 0;
144         if(search.left!=null) leftHeight = search.left.height+1;
145         int rightHeight = 0;
146         if(search.right!=null) rightHeight = search.right.height+1;
147         search.height = max(max(1,leftHeight),rightHeight);
148         search = search.middle;
149     }
150 }
151
152
153 private static int max(int a, int b){
154     return Math.max(a,b);
155 }
156
157 private static ArrayList<Integer> list = new ArrayList<>();
158 private static Stack<Nodes> stack = new Stack<>();
159 public ArrayList<Integer> getTree(){
160     list = new ArrayList<>();
161     getTree(treeHead);
162     return list;
163 }
164 private void getTree(Nodes node){
165     if(node==null) return;
166     getTree(node.left);
167     list.add(node.key);
168     getTree(node.right);
169 }
170 private static int getBalanceFactor(Nodes node){
171     if(node == null) return 0;
172     int leftHeight = 0;
173     if(node.left!=null) leftHeight = node.left.height;
174     int rightHeight = 0;
175     if(node.right!=null) rightHeight = node.right.height;
176     return leftHeight-rightHeight;
177 }
178 private void leftRotate(Nodes node){
179     // 定义父节点和冲突左节点
180     Nodes fatherNode = node.middle;
181     Nodes conflictLeftNode = node.right.left;

```

```

182 // 断开父节点和支点的联系，首先判断大小来断定左右
183 // false 代表我们选转的支点在父节点的左边
184 // true 代表我们旋转的支点在父节点的右边
185 boolean flag = (fatherNode != null && node.key > fatherNode.key);
186 if(fatherNode!=null)
187 {
188     if(flag) fatherNode.right = null;
189     else fatherNode.left = null;
190 }
191 node.middle = null;
192
193 // 首先检查支点的右节点有无左节点。要处理冲突问题
194 // 如果说没有左节点，简单左旋
195 if(conflictLeftNode==null)
196 {
197     // 断开 node 和 node.right 的联系
198     Nodes rightNode = node.right;
199     rightNode.middle = null;
200     node.right = null;
201
202     // 让fatherNode和 rightNode连接起来
203     if(fatherNode != null)
204     {
205         rightNode.middle = fatherNode;
206         if(flag)fatherNode.right = rightNode;
207         else fatherNode.left = rightNode;
208     }
209
210     // 把node和 node.right连接起来
211     node.middle = rightNode;
212     rightNode.left = node;
213
214     // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
215     if(fatherNode == null)
216         treeHead = rightNode;
217
218     // 最后，调整高度：
219     updateHeight(node);
220
221     return;
222 }
223 // 如果有左节点，复杂左旋
224 if(conflictLeftNode!=null)
225 {
226     // 断开 node 和 node.right 的联系
227     Nodes rightNode = node.right;
228     rightNode.middle = null;
229     node.right = null;
230
231     // 让fatherNode和 rightNode连接起来
232     if(fatherNode != null)
233     {
234         rightNode.middle = fatherNode;

```

```

235         if(flag) fatherNode.right = rightNode;
236         else fatherNode.left = rightNode;
237     }
238     // 切除冲突左子树
239     conflictLeftNode.middle = null;
240     rightNode.left = null;
241     // 冲突树连接到node右边
242     node.right = conflictLeftNode;
243     conflictLeftNode.middle = node;
244     // 把node和 node.right连接起来
245     node.middle = rightNode;
246     rightNode.left = node;
247
248     // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
249     if(fatherNode == null)
250         treeHead = rightNode;
251
252     // 最后，调整高度：
253     updateHeight(conflictLeftNode);
254
255
256
257     return;
258 }
259 }
260 private void rightRotate(Nodes node){
261     // 定义父节点和冲突右节点
262     Nodes fatherNode = node.middle;
263     Nodes conflictRightNode = node.left.right;
264
265     // 先判断支点在父节点的左边还是右边
266     // false 是左边， true是右边
267     boolean flag = (fatherNode!=null && node.key>fatherNode.key);
268
269     // 断开父节点和支点的联系
270     if(fatherNode!=null)
271     {
272         if(flag) fatherNode.right=null;
273         else fatherNode.left = null;
274     }
275     node.middle = null;
276
277     // 若无冲突右节点，简单右旋
278     if(conflictRightNode==null)
279     {
280         // 断开 node 和 node.left的联系
281         Nodes leftNode = node.left;
282         leftNode.middle = null;
283         node.left = null;
284
285         // 让 fatherNode 和 leftNode连起来
286         if(fatherNode!=null)
287         {

```

```

288         leftNode.middle = fatherNode;
289         if(flag) fatherNode.right = leftNode;
290         else fatherNode.left = leftNode;
291     }
292
293     // 把node和leftNode连起来
294     node.middle = leftNode;
295     leftNode.right = node;
296
297     // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
298     if(fatherNode == null)
299         treeHead = leftNode;
300
301     // 最后，调整高度：
302     updateHeight(node);
303
304     return;
305 }
306 // 如果有用冲突右节点
307 if(conflictRightNode!=null)
308 {
309     // 断开 node 和 node.left的联系
310     Nodes leftNode = node.left;
311     leftNode.middle = null;
312     node.left = null;
313
314     // 让 fatherNode 和 leftNode连起来
315     if(fatherNode!=null)
316     {
317         leftNode.middle = fatherNode;
318         if(flag) fatherNode.right = leftNode;
319         else fatherNode.left = leftNode;
320     }
321     // 切除冲突右节点
322     conflictRightNode.middle = null;
323     leftNode.right = null;
324     // 冲突右节点连接到node左节点上
325     node.left = conflictRightNode;
326     conflictRightNode.middle=node;
327     // 把node和leftNode连起来
328     node.middle = leftNode;
329     leftNode.right = node;
330
331     // 如果fatherNode == null 意味着其实node就是fatherNode，那么更改树头
332     if(fatherNode == null)
333         treeHead = leftNode;
334
335     // 最后，调整高度：
336     updateHeight(conflictRightNode);
337
338     return;
339 }
340 }

```

```

341
342     public boolean remove(int number) {
343
344         if (treeHead == null) return false; // 如果当前对象没有树，那么直接返回false，删除失
败。
345         Nodes node = treeHead;           // 让search从头节点开始操作；search代表要删除的点
346         boolean flag = false;           // search是上一个点的左边还是右边：false ->
left; true->right
347         while (node != null) {           // 找到要删除的点
348             if (number == node.key) break; // 找到就break
349             if (number > node.key) {      // 如果number大于当前search，那么往右边找。
350                 flag = true;             // flagPoint = true代表往右找
351                 node = node.right;
352                 continue;               // 一定要continue否则会下面if冲突
353             }
354             if (number < node.key) {
355                 flag = false;            // 往左边找，标记flag
356                 node = node.left;
357             }
358         }
359         // 如果不存在就返回false
360         if (node == null) return false;
361
362         // 如果要删除的是头节点，而且该对象仅有头节点，那么头节点为null即可
363         if (node == treeHead && treeHead.left == null && treeHead.right == null) {
364             treeHead = null;
365             return true;
366         }
367
368
369         // 情况1： 叶子节点
370         if (node.left == null && node.right == null) {
371             if (flag) { // 删除右边
372                 node.middle.right = null;
373             } else { // 删除左边
374                 node.middle.left = null;
375             }
376         }
377
378         // 取消依赖
379         node.middle = null;
380
381         // 取下父节点
382         Nodes fatherNode = node.middle;
383         // 更新父节点高度
384         updateHeight(fatherNode);
385         // 执行旋转(如果有)
386         executeRotateOperation(fatherNode);
387
388         return true;
389     }
390
391

```

```

392 // 情况2: 只有右子树
393 if (node.left == null && node.right != null) {
394     //如果是头节点, 直接让头节点指向下一个即可
395     if (node == treeHead) {
396         treeHead = treeHead.right;
397         treeHead.middle = null; //取消引用, 直接回收内存
398     } else if (flag) { // 右边
399         node.middle.right = node.right;
400         node.right.middle = node.middle;
401
402     } else { // 左边
403         node.middle.left = node.right;
404         node.right.middle = node.middle;
405     }
406
407     // 取下父节点
408     Nodes fatherNode = node.middle;
409     // 更新父节点高度
410     updateHeight(fatherNode);
411     // 执行旋转(如果有)
412     executeRotateOperation(fatherNode);
413
414     node.middle = null;
415     node.left = null;
416     node.right = null;
417     return true;
418 }
419 // 情况3: 只有左子树
420 if (node.left != null && node.right == null) {
421     if (node == treeHead) {
422         treeHead = treeHead.left;
423         treeHead.middle = null;
424     } else if (flag) { // 右边
425         node.middle.right = node.left;
426         node.left.middle = node.middle;
427     } else { // 左边
428         node.middle.left = node.left;
429         node.left.middle = node.middle;
430     }
431
432     // 取下父节点
433     Nodes fatherNode = node.middle;
434     // 更新父节点高度
435     updateHeight(fatherNode);
436     // 执行旋转(如果有)
437     executeRotateOperation(fatherNode);
438
439     node.middle = null;
440     node.left = null;
441     node.right = null;
442     return true;
443 }
444

```

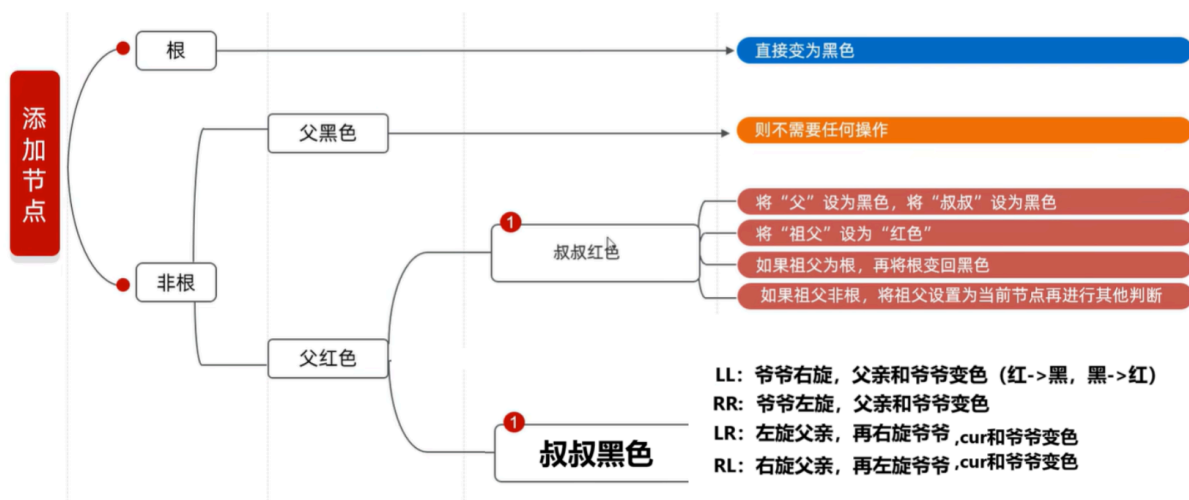
```

445 // 情况4: 既有左子树也有右子树。
446 if (node.left != null && node.right != null) {
447     boolean sign = false; // sign用与判断search节点在上一个节点的左还是右。默认
false= 左, 如果true则为右
448     Nodes temp = node; // temp是search左子树中最大的点
449     temp = temp.left;
450     while (temp.right != null) { // 让temp作为search左子树中最大的节点
451         temp = temp.right;
452         sign = true;
453     }
454
455     // 如果 temp 有左节点 , 就晋升左节点到 temp 位置
456     Nodes tempFather = temp.middle; // 记录temp的父节点
457     Nodes leftTree = temp.left; // 记录temp左子树
458
459     if (leftTree != null) leftTree.middle = null; // 左子树和temp断开
460     temp.left = null; // temp左右子树都是null (temp不可能有右子树)
461
462     // 断开temp和父节点
463     if (sign) {
464         temp.middle.right = null;
465     } else {
466         temp.middle.left = null;
467     }
468     temp.middle = null;
469
470     // 让左子树晋升
471     if (leftTree != null) {
472         leftTree.middle = tempFather;
473         if (sign) {
474             tempFather.right = leftTree;
475         } else {
476             tempFather.left = leftTree;
477         }
478     }
479
480     // 现在要让temp和search换位
481     temp.middle = node.middle;
482     temp.left = node.left;
483     temp.right = node.right;
484
485     // 如果search这个点不是treeHead, 就更改其middle
486     if (node.middle != null) {
487         if (flag) { // flagPoint 代表search是上一个点的左边还是右边, true代表search
在上一个点的右边
488             node.middle.right = temp;
489         } else { // search在上一个点的左边
490             node.middle.left = temp;
491         }
492     }
493
494     // 让search的左右子树指向temp
495     if (node.left != null) node.left.middle = temp;

```

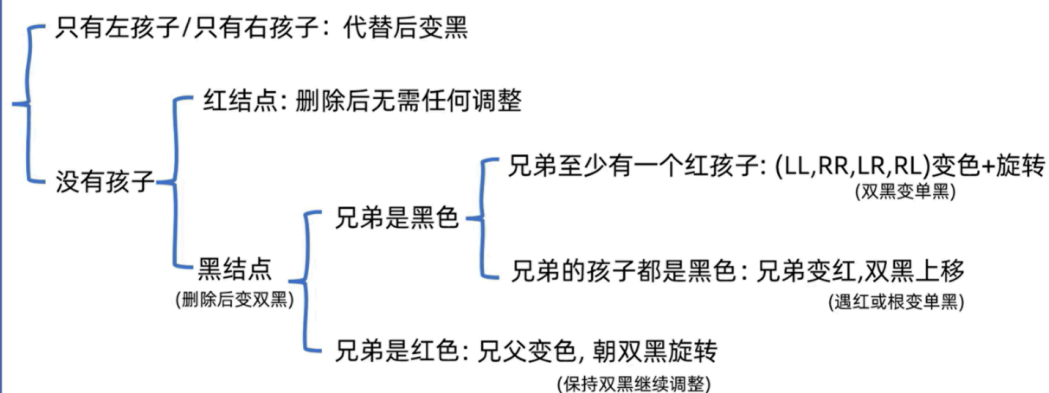
```
496         if (node.right != null) node.right.middle = temp;
497
498         // 如果更改的是树头，就让树头 = temp;
499         if (node == treeHead) treeHead = temp;
500
501         // 取下父节点
502         Nodes fatherNode = node.middle;
503         // 更新父节点高度
504         updateHeight(fatherNode);
505         // 执行旋转(如果有)
506         executeRotateOperation(fatherNode);
507
508         return true;
509     }
510
511     return false;
512 }
513
514 public boolean contains(int number){
515     Nodes node = treeHead;
516     while(node!=null)
517     {
518         if(node.key == number) return true;
519         if(number>node.key)
520         {
521             node = node.right;
522             continue;
523         }
524         if(number<node.key)
525         {
526             node = node.left;
527             continue;
528         }
529     }
530     return false;
531 }
532 }
```

红黑树



红黑树

删除



```

1  import java.util.ArrayList;
2
3  public class RedBlackTree {
4      private static final int LL = 0;
5      private static final int LR = 1;
6      private static final int RL = 2;
7      private static final int RR = 3;
8      private static int componentNumber = 0;
9      private static ArrayList<Integer> list;
10     private Nodes treeHead;
11
12     public RedBlackTree() {
13     } // 空参构造
14
15
16     // [Public] 查看是否是空树
17     public boolean isEmpty() {
18         return treeHead == null;
19     }
20
21     // [Public] 查看是否存在这个值
22     public boolean contains(int key) {
23         return findThisNode(key) != null;

```

```

24     }
25
26     // [Public] 查找并返回这个点（如果没有返回就是null）
27     private Nodes findThisNode(int key) {
28         Nodes search = treeHead;
29         while (search != null) {
30             if (key == search.key) return search;
31             if (key > search.key) {
32                 search = search.right;
33                 continue;
34             }
35             if (key < search.key) {
36                 search = search.left;
37                 continue;
38             }
39         }
40         return search;
41     }
42
43     // [Public] 插入
44     public void insert(int key) {
45         if (treeHead == null) {
46             insertTreeHead(key); // 给树头插入数据
47             componentNumber++;
48             return;
49         }
50
51         Nodes search = treeHead;
52         Nodes fatherNode = null; // 这个fatherNode 是search的父亲节点
53
54         while (search != null) {
55             if (key == search.key) return; // 存在了，不插入
56             if (key > search.key) {
57                 fatherNode = search;
58                 search = search.right;
59                 continue;
60             }
61             if (key < search.key) {
62                 fatherNode = search;
63                 search = search.left;
64                 continue;
65             }
66         }
67         // 执行完之后，search就是我们要插入数据的地方
68         search = new Nodes(); // 申请新节点
69         search.key = key; // 更新数值
70         connectNodes(fatherNode, search);
71         componentNumber++;
72
73         rule(search); // 按照红黑树规则更新节点
74     }
75
76     // [Public] 返回这棵树

```

```

77     public ArrayList<Integer> getTree() {
78         list = new ArrayList<>();
79         getTree(treeHead);
80         return list;
81     }
82     // [Public] 获取这棵树元素个数
83
84     public int getComponentNumber() {
85         return componentNumber;
86     }
87
88     // [Public] 删除
89     public void delete(int key) {
90         Nodes node = findThisNode(key);
91         if (node == null) return; // 如果这个节点不存在，就不用删除
92         delete(node);
93         componentNumber--;
94     }
95
96     // 函数重载用于内部
97     // 传入节点不可能为null
98     private void delete(Nodes node) {
99         // 特殊情况删除树头
100         if (leaf(node) && node == treeHead) {
101             treeHead = null;
102             return;
103         }
104
105         // 如果这个节点是叶子节点（复杂情况）
106         if (leaf(node)) {
107             deleteOperationA(node);
108             return;
109         }
110         // 如果这个节点只有singleBranch，执行操作B
111         if (singleBranch(node)) {
112             deleteOperationB(node);
113             return;
114         }
115         // 如果运行到这里就证明这个节点左右都有节点
116         deleteOperationH(node);
117     }
118
119
120     private void deleteOperationH(Nodes node) {
121         Nodes min = findMinNode(node.right); // 找左边最大的 or 右边最小的
122         delete(min);
123         copyKeyProperties(node, min);
124     }
125
126     // 把 source中的所有属性覆盖到target上（除了节点left,right,middle和color，这个是属于树的属性）
127     // 如果以后Node 更新了更多属性例如<E>那么这里就需要更改
128     private void copyKeyProperties(Nodes target, Nodes source) {

```

```

129     target.key = source.key;
130 }
131
132 // 寻找当前节点下最大的节点（包括自身）
133 private Nodes findMaxNode(Nodes node) {
134     Nodes finalReturn = node;
135     while (finalReturn.right != null) finalReturn = finalReturn.right;
136     return finalReturn;
137 }
138
139 // 寻找当前节点下最小的节点（包括自身）
140 private Nodes findMinNode(Nodes node) {
141     Nodes finalReturn = node;
142     while (finalReturn.left != null) finalReturn = finalReturn.left;
143     return finalReturn;
144 }
145
146 // 这个节点只有左边子树或者右边子树。
147 // 直接晋升孩子+转孩子为黑色
148 private void deleteOperationB(Nodes node) {
149     // 只有左子树
150     if (node.left != null) {
151         Nodes middleNode = node.middle;
152         Nodes leftNode = node.left;
153
154         disconnectNodes(middleNode, node);
155         disconnectNodes(node, leftNode);
156
157         if (node == treeHead) treeHead = leftNode;
158         else connectNodes(middleNode, leftNode);
159
160         leftNode.color = false; // 晋升节点需要转为黑色
161     } else { // 只有右子树
162         Nodes middleNode = node.middle;
163         Nodes rightNode = node.right;
164
165         disconnectNodes(middleNode, node);
166         disconnectNodes(node, rightNode);
167
168         if (node == treeHead) treeHead = rightNode;
169         else connectNodes(middleNode, rightNode);
170         rightNode.color = false; // 晋升节点需要转为黑色
171     }
172     // componentNumber--;
173 }
174
175 // 如果这个节点是singleBranch的，则返回True否则返回false
176 private boolean singleBranch(Nodes node) {
177     return (node.left == null && node.right != null) || (node.left != null &&
node.right == null);
178 }
179
180 // 如果这个节点是叶子节点（没有孩子）

```

```

181 private void deleteOperationA(Nodes node) {
182     if (getColor(node)) // 如果是红节点，直接删除
183     {
184         Nodes fatherNode = node.middle;
185         disconnectNodes(fatherNode, node);
186         // componentNumber -- ;
187         return;
188     }
189     // 接下来，是删除黑色叶子节点的操作
190     deleteOperationD(node, true); // 从这里传入的node不可能是null
191 }
192
193 // 这个操作仅仅用于删除黑色节点(或调整)，而且这个黑色节点是叶子节点（无左右节点）
194 private void deleteOperationD(Nodes node, boolean delete) {
195     if (node == treeHead && !delete) // 如果传递到树头，而且是调整模式（不删除），直接转为
黑色return
196     {
197         node.color = false;
198         return;
199     }
200     Nodes brotherNode = getBrother(node); // 这个brotherNode可能是null
201     if (brotherNode == null) {
202         return;
203     }
204     if (getColor(brotherNode)) {
205         // 兄弟节点是红色的
206         deleteOperationG(node, brotherNode, delete);
207         return;
208     }
209     // 这个兄弟节点是黑色的
210
211     if (atLeastOneRedChild(brotherNode))
212         // 如果兄弟至少有一个红色孩子，就执行deleteOperationE
213         deleteOperationE(node, brotherNode, delete);
214     else
215         // 反之如果兄弟都是黑色孩子，就执行deleteOperationF
216         deleteOperationF(node, brotherNode, delete);
217 }
218
219 private void deleteOperationG(Nodes node, Nodes brotherNode, boolean delete) {
220     // 兄弟父亲变色
221     switchColor(brotherNode);
222     switchColor(node.middle);
223
224     Nodes fatherNode = node.middle;
225     boolean rotate = getPosition(fatherNode, node);
226     // 父节点朝着node方向旋转
227     if (rotate) rightRotateNode(fatherNode);
228     else leftRotateNode(fatherNode);
229
230     deleteOperationD(node, false); // 要删除的节点是node，但现在先保留，继续调整
231     // 调整完成后，就删除
232     if (delete) {

```

```

233         // componentNumber--;
234         disconnectNodes(fatherNode, node);
235     }
236 }
237
238 // 传入brother节点
239 private void deleteOperationE(Nodes node, Nodes brotherNode, boolean delete) {
240     switch (getPattern(brotherNode)) {
241         case LL -> {
242             // 颜色更新
243             brotherNode.left.color = getColor(brotherNode);
244             brotherNode.color = getColor(brotherNode.middle);
245             brotherNode.middle.color = false;
246             // 右旋转父节点
247             rightRotateNode(node.middle);
248             // 删除目标节点
249             if (delete) {
250                 // componentNumber--;
251                 disconnectNodes(node.middle, node);
252             }
253         }
254         case RR -> {
255             // 颜色更新
256             brotherNode.right.color = getColor(brotherNode);
257             brotherNode.color = getColor(brotherNode.middle);
258             brotherNode.middle.color = false;
259             // 左旋转父节点
260             leftRotateNode(node.middle);
261             // 删除目标节点
262             if (delete) {
263                 // componentNumber--;
264                 disconnectNodes(node.middle, node);
265             }
266         }
267         case LR -> {
268             // 颜色更新
269             brotherNode.right.color = getColor(brotherNode.middle);
270             brotherNode.middle.color = false;
271             // 左旋 brother
272             leftRotateNode(brotherNode);
273             // 右旋父节点
274             rightRotateNode(node.middle);
275             // 删除目标节点
276             if (delete) {
277                 // componentNumber--;
278                 disconnectNodes(node.middle, node);
279             }
280         }
281         case RL -> {
282             brotherNode.left.color = getColor(brotherNode.middle);
283             brotherNode.middle.color = false;
284             rightRotateNode(brotherNode);
285             leftRotateNode(node.middle);

```

```

286         if (delete) {
287             // componentNumber--;
288             disconnectNodes(node.middle, node);
289         }
290     }
291 }
292 }
293
294 private void deleteOperationF(Nodes node, Nodes brotherNode, boolean delete) {
295     brotherNode.color = true; // 兄弟变红
296     Nodes fatherNode = node.middle;
297     boolean fatherNodeColor = getColor(fatherNode);
298     if (delete) {
299         // componentNumber--;
300         disconnectNodes(fatherNode, node);
301     }
302     if (fatherNodeColor) // 双黑节点上移,
303         fatherNode.color = false; // 如果双黑移动到红色节点, 直接变黑。结束
304     else deleteOperationD(fatherNode, false); // 不删除只调整
305 }
306
307 // 返回四种形态之一, 传入的node必须为中间节点
308 private int getPattern(Nodes node) {
309     if (getPosition(node.middle, node) && getColor(node.right)) return RR;
310
311     if (!getPosition(node.middle, node) && getColor(node.left)) return LL;
312
313     if (getPosition(node.middle, node) && !getColor(node.right) &&
314         getColor(node.left)) return RL;
315
316     if (!getPosition(node.middle, node) && getColor(node.right) &&
317         !getColor(node.left)) return LR;
318
319     System.err.println("NO PATTERN FOUND!");
320     return -1;
321 }
322
323 // 如果这个节点至少有一个孩子是红色, 就返回true
324 // 也就是如果俩孩子都是黑色就返回false
325 private boolean atLeastOneRedChild(Nodes node) {
326     return (node.left != null && node.left.color) || (node.right != null &&
327         node.right.color);
328 }
329
330 // 返回这个节点的兄弟节点
331 private Nodes getBrother(Nodes node) {
332     // 特殊
333     if (node.middle != null && node.key == node.middle.key) {
334         if (node.middle.right == node) return node.middle.left;
335         else return node.middle.right;
336     }
337     // 一般
338     if (getPosition(node.middle, node)) return node.middle.left;

```

```

336         else return node.middle.right;
337     }
338
339     // 如果这个节点是单节点就返回 True, 否则返回false
340     private boolean leaf(Nodes node) {
341         if (node == null) return false;
342         return node.left == null && node.right == null;
343     }
344
345     private void getTree(Nodes nodes) {
346         if (nodes == null) return;
347         getTree(nodes.left);
348         list.add(nodes.key);
349         getTree(nodes.right);
350     }
351
352     // 按照红黑树规则更新 node 节点
353     private void rule(Nodes node) {
354         if (node == treeHead) return; // 如果是根节点, 不执行任何操作
355
356         // 接下来是非根节点
357         if (!getColor(node.middle)) return; // 如果父节点是黑色, 不执行任何操作
358
359         // 接下来是非根节点且父节点是红色
360
361         Nodes uncleNode = getUncleNodes(node); // 叔叔节点
362         if (getColor(uncleNode)) // 如果叔叔节点是红色, 执行optionA。需要考虑递归
363         {
364             insertOperationA(node);
365             return;
366         }
367         // 接下来是非根节点且父节点是红色, 叔叔节点是黑色
368         Nodes fatherNode = node.middle;
369         Nodes grandfatherNode = node.middle.middle;
370         switch (getPattern(node.middle)) {
371             case LL -> {
372                 rightRotateNode(grandfatherNode); // 爷爷右旋
373                 switchColor(fatherNode); // 转变颜色 适用于LL和RR
374                 switchColor(grandfatherNode);
375             }
376             case RR -> {
377                 leftRotateNode(grandfatherNode);
378
379                 switchColor(fatherNode);
380                 switchColor(grandfatherNode);
381             }
382             case LR -> {
383                 leftRotateNode(fatherNode); // 左旋父亲
384                 rightRotateNode(grandfatherNode); // 右旋爷爷
385
386                 switchColor(node); // 转变颜色 适用于LR和RL
387                 switchColor(grandfatherNode);
388             }

```

```

389         case RL -> {
390             rightRotateNode(fatherNode);
391             leftRotateNode(grandfatherNode);
392
393             switchColor(node);
394             switchColor(grandfatherNode);
395         }
396     }
397
398 }
399
400 private void switchColor(Nodes node) {
401     if (node != null) node.color = !node.color;
402 }
403
404 // 左旋转node
405 private void leftRotateNode(Nodes node) {
406     Nodes fatherNode = node.middle;
407     Nodes leftNode = node.left;
408     Nodes rightNode = node.right;
409
410
411     Nodes subNode = null; // 这个是冲突的节点（如果不为null）
412     if (rightNode != null) subNode = rightNode.left;
413
414     disconnectNodes(node, rightNode);
415     disconnectNodes(fatherNode, node);
416     disconnectNodes(rightNode, subNode);
417
418
419     connectNodes(rightNode, node);
420     connectNodes(node, subNode);
421     if (node != treeHead) // 如果 node 不是 treeHead, 那么就链接
422         connectNodes(fatherNode, rightNode);
423     else treeHead = rightNode; // 如果 node 是treeHead, 那么就链接给 rightNode
424 }
425
426 // 右旋转node
427 private void rightRotateNode(Nodes node) {
428     Nodes fatherNode = node.middle;
429     Nodes leftNode = node.left;
430     Nodes rightNode = node.right;
431
432
433     Nodes subNode = null;
434     if (leftNode != null) subNode = leftNode.right;
435
436     disconnectNodes(node, leftNode);
437     disconnectNodes(fatherNode, node);
438     disconnectNodes(leftNode, subNode);
439
440
441     connectNodes(leftNode, node);

```

```

442     connectNodes(node, subNode);
443     if (node != treeHead) // 如果 node 不是 treeHead, 那么就链接
444         connectNodes(fatherNode, leftNode);
445     else treeHead = leftNode; // 如果 node 是 treeHead, 那么就链接 leftNode
446 }
447
448 // 如果新插入节点是非根节点, 而且父节点是红色, 叔叔节点是红色, 执行OptionA
449 // 传入的Node是插入节点
450 // 返回值如果是true代表仍然需要递归。false表示可以结束
451 private void insertOperationA(Nodes node) {
452     // 把父节点设置为黑色
453     node.middle.color = false;
454     // 把叔叔节点设置为黑色
455     getUncleNodes(node).color = false;
456     // 把爷爷设置为红色
457     node.middle.middle.color = true;
458     // 如果爷爷是根节点, 就把爷爷设置为黑色
459     if (node.middle.middle == treeHead) node.middle.middle.color = false;
460     else // 如果爷爷不是根节点, 就需要递归重新判断, 判断对象是爷爷节点
461         rule(node.middle.middle);
462 }
463
464 // 获取叔叔节点
465 private Nodes getUncleNodes(Nodes node) {
466     if (node.middle.middle == null) {
467         System.out.println();
468     }
469     boolean position = getPosition(node.middle.middle, node.middle);
470     if (position) // 如果node的父节点是node的爷爷节点的右节点, 那么uncle节点就
是.middle.middle.left
471         return node.middle.middle.left;
472     else // 如果node的父节点是node的爷爷节点的左节点, 那么uncle节点就
是.middle.middle.right
473         return node.middle.middle.right;
474 }
475
476 // 取消这两个点的链接, 第一个传入的是父亲节点(强制), 第二个传入的是子节点
477 private void disconnectNodes(Nodes A, Nodes B) {
478     // 特殊情况
479     if (A != null && B != null && (A.key == B.key)) {
480         B.middle = null;
481         if (A.right == B) A.right = null;
482         else A.left = null;
483         return;
484     }
485     // 一般情况
486     if (A == null || B == null) return;
487     if (getPosition(A, B)) A.right = null;
488     else A.left = null;
489     B.middle = null;
490 }
491
492 // 链接这两个节点, 第一个传入的是父亲节点(强制), 第二个传入的是子节点

```

```

493 private void connectNodes(Nodes A, Nodes B) {
494     if (A == null || B == null) return;
495     if (getPosition(A, B)) A.right = B;
496     else A.left = B;
497     B.middle = A;
498 }
499
500 // 返回position信息，第一个传入的是父亲节点（强制），第二个传入的是子节点
501 // true 代表子节点在父节点右边
502 // false 代表子节点在父节点左边
503 private boolean getPosition(Nodes A, Nodes B) {
504     return B.key > A.key;
505 }
506
507 private void insertTreeHead(int key) {
508     treeHead = new Nodes();
509     treeHead.key = key;
510     treeHead.color = false; // 根节点必须颜色是黑色（false）的
511 }
512
513 // getColor 能处理Nil节点（null节点），返回false（黑色）
514 // 如果不是Nil节点，就返回节点默认颜色
515 private boolean getColor(Nodes node) {
516     if (node == null) return false;
517     return node.color;
518 }
519
520 private static class Nodes {
521     int key;
522     Nodes left;
523     Nodes right;
524     Nodes middle;
525     boolean color = true; // true代表这个节点是红色
526 }
527 }

```